Machine Intelligence II
Lecture Notes

Arjun Puri

Summer 2026

Contents

Preface

These notes cover the second Machine Intelligence sequence, beginning with unsupervised representation learning. The first lectures use principal component analysis and Hebbian learning as a controlled setting where geometry, optimization, and online adaptation can be studied without labels.

The organizing question is how structure in data becomes structure in a representation. PCA gives the batch spectral answer: diagonalize covariance and keep the directions that explain variance. Hebbian and anti-Hebbian rules give the online dynamical answer: update weights from local activity so the network tracks useful directions as samples arrive.

Berlin, April 2026

Notation

Data and samples

Symbol	Meaning
$x^{(\alpha)}$	Observation α in feature space
N	Number of observed features
m	Sample mean of the dataset
C	Sample covariance matrix

Principal Component Analysis

Symbol	Meaning
e_a	Principal direction / eigenvector
λ_a	Variance captured by principal direction e_a
z_a	Coordinate of the data along component a
M	Retained number of principal components

Learning paradigms

Term	Meaning
Supervised learning	Learn targets from labeled examples
Reinforcement learning	Learn action policies from rewards and interaction
Unsupervised learning	Learn useful structure from unlabeled observations

1 Lecture 1: Principal Component Analysis

Principal Component Analysis (PCA), also known as the Karhunen–Loeve transform, is the first projection method in Machine Intelligence II. The problem starts with observations

$$x^{(\alpha)} \in \mathbb{R}^N, \qquad \alpha = 1, \ldots, p,$$

and asks which directions in feature space expose the relevant structure of the data. PCA answers this question with a variance criterion: directions are interesting when the data vary strongly along them.

1.1 Projection Methods and Data Structure

The lecture contrasts two families of unsupervised methods. Projection methods search for informative directions in feature space, while clustering methods search for groups, categories, or prototypes. PCA belongs to the projection family. It does not first ask which class an observation belongs to; it asks which coordinate system makes the data cloud easiest to understand.

From raw variables to directions. An elementary feature is one coordinate axis of the original measurement space. A complex feature is a direction e_a in that same space. Its value on an observation is the scalar projection

$$u_a(x) = e_a^{\mathsf{T}} x.$$

PCA is a principled way to choose such directions from the data.

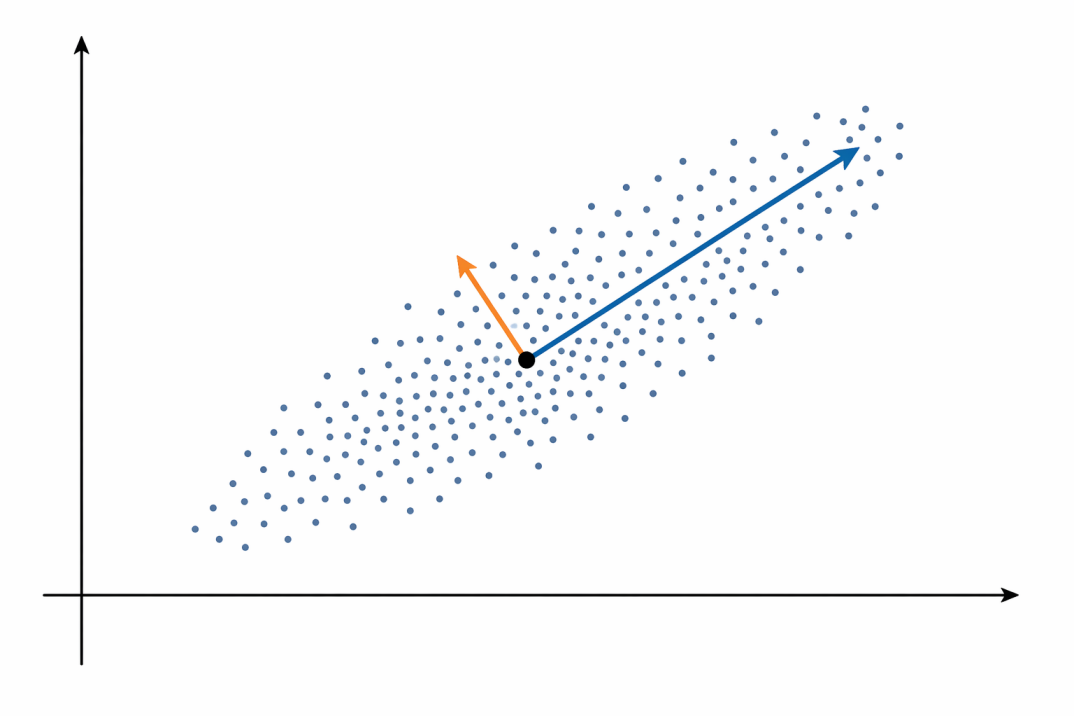

Figure 1: A data cloud has different variances in different directions. PCA chooses the first direction along the largest spread of the centered cloud, then chooses orthogonal directions that explain the remaining variance.

The iris example in the slides is a useful starting intuition. A flower is measured by several numerical attributes, and scatter plots of pairs of attributes already show some structure. PCA generalizes this search beyond pairwise plots: it finds linear combinations of the original attributes that can reveal separation, compression, or nuisance variation.

1.2 Moments of the Data Cloud

PCA is built from the first two empirical moments of the observations. The sample mean is the center of mass,

$$m = \frac{1}{p} \sum_{\alpha=1}^{p} x^{(\alpha)}.$$

The covariance matrix records the centered second moments:

$$C_{ij} = \frac{1}{p} \sum_{\alpha=1}^{p} (x_i^{(\alpha)} - m_i)(x_j^{(\alpha)} - m_j).$$

For centered data, where $m = 0$, this simplifies to

$$C_{ij} = \frac{1}{p} \sum_{\alpha=1}^{p} x_i^{(\alpha)} x_j^{(\alpha)}.$$

The diagonal entries C_{ii} are variances of individual variables. The off-diagonal entries C_{ij} are covariances between variables. The matrix is symmetric, so $C_{ij} = C_{ji}$. A zero covariance means two variables are uncorrelated, but it does not prove statistical independence; nonlinear dependence can remain invisible to a second-order method.

1.3 Variance Along a Complex Feature

Let e_a be a unit vector in feature space. The induced feature value is

$$u_a^{(\alpha)} = e_a^{\mathsf{T}} x^{(\alpha)}.$$

Its mean is

$$m_a = \frac{1}{p} \sum_{\alpha=1}^{p} u_a^{(\alpha)} = e_a^{\mathsf{T}} m.$$

Its variance is

$$\sigma_a^2 = \frac{1}{p} \sum_{\alpha=1}^{p} \left(u_a^{(\alpha)} - m_a\right)^2 = e_a^{\mathsf{T}} C e_a.$$

This identity is the hinge of the lecture: the covariance matrix determines the variance of the data along every possible direction. Once C is known, PCA can compare all candidate directions without returning to the raw data points.

1.4 The Principle of Maximal Variance

The first principal component is the unit direction with maximal projected variance:

$$e_1 = \arg\max_{\|e\|=1} e^{\mathsf{T}} C e.$$

The unit-length constraint is essential. Without it, the variance could be made arbitrarily large by scaling e. Introducing a Lagrange multiplier gives

$$\mathcal{L}(e, \lambda) = e^{\mathsf{T}} C e - \lambda(e^{\mathsf{T}} e - 1).$$

Setting the derivative with respect to e to zero yields

$$Ce = \lambda e.$$

Thus the principal components are the normalized eigenvectors of the covariance matrix. The variance along a principal component is the corresponding eigenvalue:

$$\sigma_a^2 = e_a^{\mathsf{T}} C e_a = \lambda_a.$$

PCA eigenproblem.

$$Ce_a = \lambda_a e_a, \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N.$$

The eigenvectors give the principal directions. The eigenvalues rank those directions by explained variance.

1.5 Properties of Principal Components

Because the covariance matrix is real and symmetric, its eigenvectors form an orthonormal basis:

$$e_i^{\mathsf{T}} e_j = \delta_{ij}.$$

Ordering the eigenvalues from largest to smallest orders the corresponding eigenvectors from most variable to least variable. The first principal component points along the largest variance in the full space. The second points along the largest remaining variance in the subspace orthogonal to the first, and so on.

1.5.1 Optimal Linear Dimensionality Reduction

Any observation can be represented in the principal-component basis:

$$x = a_1 e_1 + a_2 e_2 + \cdots + a_N e_N, \qquad a_j = e_j^{\mathsf{T}} x.$$

Keeping only the first M terms gives the rank-M PCA approximation

$$\tilde{x} = a_1 e_1 + a_2 e_2 + \cdots + a_M e_M.$$

Among all M-dimensional linear projections, this choice minimizes the mean-squared reconstruction error:

$$E = \frac{1}{p} \sum_{\alpha=1}^{p} \|x^{(\alpha)} - \tilde{x}^{(\alpha)}\|^2 = \sum_{j=M+1}^{N} \lambda_j.$$

In words, the error is the variance left behind in the discarded principal components.

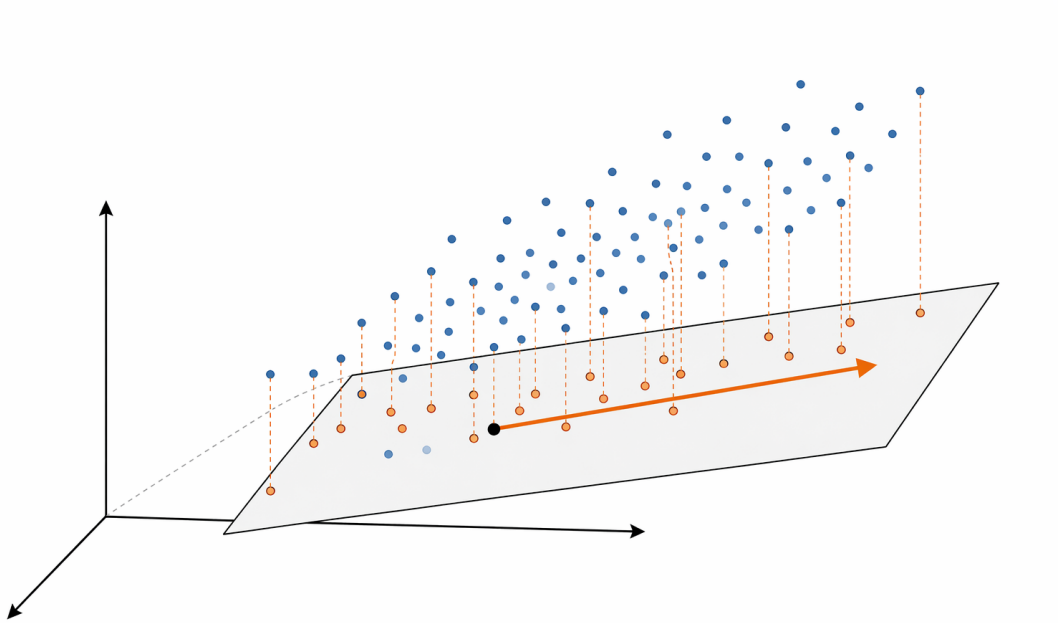

Figure 2: Dimensionality reduction by projection. PCA keeps the principal subspace that preserves the largest variance and leaves only the orthogonal residual as reconstruction error.

1.5.2 Decorrelation

Let

$$M = (e_1, e_2, \ldots, e_N)$$

be the matrix whose columns are the principal components. In this basis, the covariance matrix is diagonal:

$$M^{\mathsf{T}} C M = \Lambda = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_N).$$

The transformed features are therefore uncorrelated. This is why PCA is often used before regression, classification, clustering, or visualization: it rotates the coordinate system into axes that separate second-order variation cleanly.

1.5.3 Whitening

Variance is scale sensitive. If one input dimension is measured in much larger units than another, a variance criterion may prioritize scale rather than structure. PCA therefore only makes direct sense when feature scales are comparable, or after suitable normalization.

After decorrelation, whitening rescales every principal direction to unit variance:

$$v^{(\alpha)} = \Lambda^{-1/2} M^{\mathsf{T}} x^{(\alpha)}.$$

The result has covariance equal to the identity matrix. Whitening removes both linear correlations and unequal variances, making the transformed representation scale-standardized.

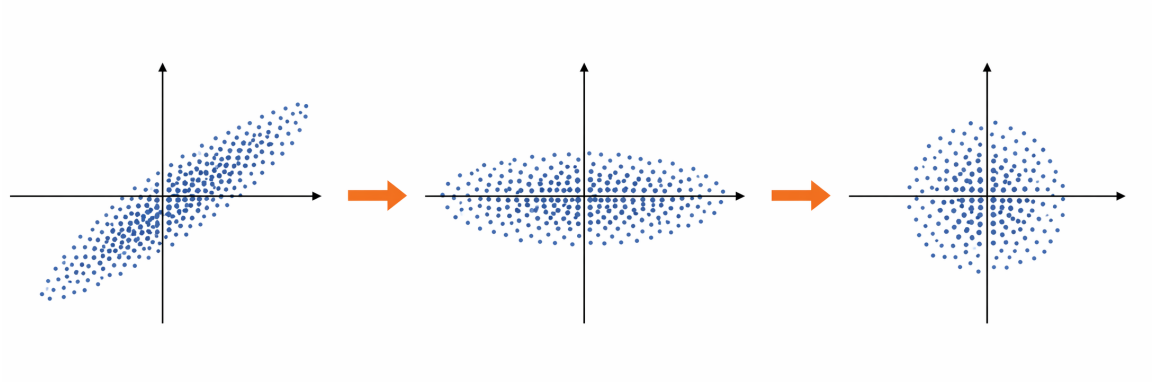

Figure 3: PCA first rotates an elongated data cloud into its eigenbasis, where the covariance is diagonal. Whitening then rescales the axes so the transformed cloud has unit variance in every retained direction.

1.6 Applications and Interpretations

1.6.1 Iris Data

The iris slides use a familiar biological dataset to motivate projection methods. Pairwise scatter plots can show partial separation between setosa, versicolor, and virginica, but PCA asks for the strongest directions in the full feature space rather than inspecting every pair of raw features. This makes it a compact exploratory tool when there are more variables than can be inspected directly.

1.6.2 Leptograpsus Crab Measurements

The Leptograpsus example is more explicitly about complex features. Each crab is described by five measurements: frontal lobe size, rear width, carapace length, carapace width, and body depth. PCA forms linear combinations of these elementary features. The leading component can capture overall size, while later components may align with sex- or species-related variation if those factors explain substantial covariance structure.

This example also warns against overinterpreting components. A principal component is a direction chosen by variance, not by semantics. A component may become interpretable as “size” or “sex” only after examining its loadings and how observations project onto it.

1.6.3 Outlier Detection

The lecture also points out a less obvious use of PCA: components with the smallest eigenvalues can help detect outliers or novel features. A normal data point should not vary much along a low-variance direction. A large projection onto such a direction can therefore indicate an unusual observation, a measurement artifact, or a rare structure not captured by the dominant modes.

1.7 Computational Notes and Extensions

PCA is linear, second-order, and variance-based. This is both its strength and its limitation. It is efficient, interpretable, and useful for preprocessing, dimensionality reduction, and compression. But very large covariance matrices can be numerically unstable or expensive, and second-order structure cannot capture every kind of dependence.

The slide summary points to two algorithmic routes for extracting leading principal components: Hebbian learning and the power method. Both are useful when the largest eigenvectors matter more than the full eigenspectrum. It also points forward to extensions. Kernel PCA replaces the raw feature space with nonlinear features, while probabilistic PCA and factor analysis add an explicit generative model. Later methods such as ICA change the objective from decorrelation to statistical independence.

Summary. PCA begins with the covariance matrix and asks which unit directions maximize projected variance. The solution is the eigenbasis of C: eigenvectors are principal components, and eigenvalues measure explained variance. Keeping the leading components gives the optimal linear low-dimensional reconstruction, rotating into the eigenbasis decorrelates the features, and whitening additionally rescales them to unit variance. The method is simple and powerful, but it remains linear and variance-based, so later MI2 methods relax or extend these assumptions.

2 Lecture 2: Online PCA and Hebbian Learning

The second lecture starts from the PCA eigenproblem and asks a more biological and algorithmic question: can the leading principal directions be learned incrementally, without first forming and diagonalizing the full covariance matrix?

The answer is yes, but each part of that statement matters.

A *linear neuron* is just a weighted sum $y = w^{\mathsf{T}}x$. It turns a high-dimensional input x into one number y, the projection of the input onto the current weight direction w. If w points along a high-variance direction of the data, then the output y varies a lot across examples.

A *Hebbian update* strengthens the weights in the direction of inputs that already produce a large output. Averaged over many samples, this update multiplies the weight vector by the covariance matrix. That is why Hebbian learning resembles the *power method*: the power method repeatedly multiplies a vector by a matrix, and repeated multiplication by a covariance matrix amplifies the eigenvector with the largest eigenvalue.

The plain Hebbian rule has one missing ingredient: it controls direction but not length. The weight vector keeps growing. Oja's rule adds a normalization term, so the direction can rotate toward the first principal component while the length stays bounded. Sanger's rule repeats the same idea with several neurons and subtracts off directions already learned, so later neurons learn later principal components. Reversing the sign gives anti-Hebbian learning: directions with large variance are suppressed, leaving small-variance directions that can be used as novelty filters.

2.1 PCA As A Learning Problem

For centered observations $x^{(\alpha)} \in \mathbb{R}^N$, the covariance matrix is

$$C = \frac{1}{p}\sum_{\alpha=1}^{p} x^{(\alpha)}x^{(\alpha)\mathsf{T}}.$$

Centered means that the sample mean has already been subtracted, so the origin represents the average input. Each term $x^{(\alpha)}x^{(\alpha)\mathsf{T}}$ is an outer product. It records which coordinates of the input tend to be large together. The covariance matrix is therefore a summary of the shape of the data cloud.

PCA asks for the unit vector e that maximizes projected variance:

$$e_1 = \arg\max_{\|e\|=1} e^\mathsf{T} C e.$$

The phrase *projected variance* means: choose a direction e, drop every data point onto that line, and measure how spread out the resulting one-dimensional numbers are. PCA chooses the line where that spread is largest.

The stationary condition is the eigenvalue equation

$$Ce = \lambda e.$$

An eigenvector is a direction that the matrix C stretches but does not rotate. The eigenvalue λ is the amount of stretch. For a covariance matrix, that stretch is variance along the corresponding direction. So the first principal component is the eigenvector with the largest eigenvalue.

The offline solution computes C and diagonalizes it. Online PCA instead sees one pattern at a time and updates a weight vector. The data stream is turned into a dynamical system: as examples arrive, the vector w moves. The goal is to design the movement so that the stable direction of w is the first principal component.

2.2 A Linear Neuron

The model neuron in the lecture is deliberately simple:

$$y = w^\mathsf{T} x.$$

This is not meant to be a detailed biophysical neuron. It is the minimal algebraic model needed for PCA. The input x is a centered observation, w is the synaptic weight vector, and y is the scalar output. The output is large when the input points in a similar direction to the current weight vector.

This model is useful because the same expression appears in PCA. The projection of x onto a candidate direction w is exactly $w^\mathsf{T} x$. Learning w is therefore learning a projection direction.

2.3 Hebb's Rule Finds The Dominant Direction

Hebbian learning is summarized by the phrase: units that are active together should strengthen their connection. For the linear neuron, the update is

$$\Delta w = \varepsilon y x,$$

or componentwise,

$$\Delta w_j = \varepsilon y x_j.$$

The learning rate $\varepsilon > 0$ is small. Each sample nudges the weights in the direction of the current input, scaled by the output. Inputs that already align with w produce larger outputs and therefore larger changes.

This rule is local in the neural sense: the change in weight w_j depends only on the presynaptic activity x_j, the postsynaptic activity y, and the learning rate. No global covariance matrix has to be stored in the neuron.

To see why this is PCA, average the update over many centered samples:

$$\mathbb{E}[\Delta w] = \varepsilon \mathbb{E}[yx] = \varepsilon \mathbb{E}[xx^{\mathsf{T}}]w = \varepsilon C w.$$

The mean update is multiplication by the covariance matrix. This is the key bridge to PCA.

The power method is a simple eigenvector algorithm. Start with almost any vector w, repeatedly apply the matrix C, and renormalize:

$$w \leftarrow \frac{Cw}{\|Cw\|}.$$

The component of w along the largest-eigenvalue direction is stretched the most, so after many steps the vector points toward the first eigenvector. Hebbian learning is a noisy, sample-by-sample version of the same idea: it does not apply C exactly, but its average update is Cw.

Let $e_1, \ldots, e_N$ be the eigenvectors of C, with eigenvalues

$$\lambda_1 > \lambda_2 > \cdots > \lambda_N.$$

Write the weight vector in this eigenbasis:

$$w = a_1 e_1 + a_2 e_2 + \cdots + a_N e_N.$$

Since $Ce_j = \lambda_j e_j$, the averaged update gives

$$\Delta a_j = \varepsilon \lambda_j a_j.$$

The coefficient a_j says how much of w points along eigenvector e_j. The coefficient along e_1 grows fastest because λ_1 is largest. If the initial weight vector has any nonzero component in the e_1 direction, the direction of w approaches e_1.

The problem is the norm. Pure Hebbian learning does not keep $\|w\|$ fixed. The direction becomes useful, but the weight length diverges. PCA cares about directions, not arbitrary vector length, so a useful online rule must include some form of normalization.

2.4 Oja's Rule Adds The Missing Constraint

PCA is a constrained optimization problem: the direction e must have unit norm. Without the unit-norm constraint, the objective $e^{\mathsf{T}} C e$ could be made arbitrarily large just by scaling e. Oja's rule builds that constraint into the learning rule:

$$\Delta w = \varepsilon y(x - yw).$$

In components,

$$\Delta w_j = \varepsilon y(x_j - y w_j).$$

The first term, εyx, is the Hebbian attraction term. The second term, $-\varepsilon y^2 w$, is a stabilizing decay term that grows when the output is large. It subtracts from the current weight direction, so it prevents the vector from increasing without bound.

The derivation below shows where the second term comes from. The key idea is simple: take an ordinary Hebbian step, then explicitly normalize the weight vector back to unit length. Oja's rule is the first-order approximation to that normalized update.

2.5 Deriving Oja's Rule From Explicit Normalization

Consider one learning step at time t, using one centered input pattern $x^{(\alpha)}$. To keep the notation readable, write

$$w = w(t), \qquad x = x^{(\alpha)}, \qquad y = y^{(\alpha)} = w^{\mathsf{T}} x.$$

Assume the current weight vector has already been normalized:

$$\|w\| = 1.$$

This is the invariant that explicit normalization is trying to maintain.

Step 1. Take the Hebbian step. The unnormalized Hebbian update is

$$\tilde{w} = w + \varepsilon y x.$$

This step points the weight vector toward the current input x, with strength proportional to the current output y.

Step 2. Normalize the result. Instead of keeping $\tilde{w}$, divide it by its Euclidean length:

$$w(t+1) = \frac{\tilde{w}}{\|\tilde{w}\|}.$$

This guarantees $\|w(t+1)\| = 1$ exactly. The question is what local update this corresponds to when the learning rate ε is small.

Step 3. Expand the squared norm. Compute the denominator:

$$\begin{aligned} \|\tilde{w}\|^2 &= (w + \varepsilon y x)^{\mathsf{T}} (w + \varepsilon y x) \\ &= w^{\mathsf{T}} w + 2\varepsilon y w^{\mathsf{T}} x + \varepsilon^2 y^2 x^{\mathsf{T}} x. \end{aligned}$$

Since $\|w\| = 1$ and $y = w^{\mathsf{T}} x$, this becomes

$$\|\tilde{w}\|^2 = 1 + 2\varepsilon y^2 + \varepsilon^2 y^2 \|x\|^2.$$

The last term is order ε^2. For a small learning rate, it is much smaller than the order-ε term, so

$$\|\tilde{w}\|^2 = 1 + 2\varepsilon y^2 + \mathcal{O}(\varepsilon^2).$$

Step 4. Expand the reciprocal norm. The normalized update needs $1/\|\tilde{w}\|$, not $\|\tilde{w}\|^2$. Use the Taylor expansion

$$(1+z)^{-1/2} = 1 - \frac{1}{2} z + \mathcal{O}(z^2) \qquad \text{for small } z.$$

Here $z = 2\varepsilon y^2 + \mathcal{O}(\varepsilon^2)$, so

$$\frac{1}{\|\tilde{w}\|} = 1 - \varepsilon y^2 + \mathcal{O}(\varepsilon^2).$$

This is the whole normalization correction. Dividing by the length is approximately the same as multiplying by $1 - \varepsilon y^2$.

Step 5. Multiply out the normalized update. Substitute the approximation into the normalized weight:

$$\begin{aligned} w(t+1) &= \frac{w + \varepsilon yx}{\|\tilde{w}\|} \\ &= (w + \varepsilon yx)\left(1 - \varepsilon y^2 + \mathcal{O}(\varepsilon^2)\right). \end{aligned}$$

Multiplying and dropping terms of order ε^2 gives

$$w(t+1) = w + \varepsilon yx - \varepsilon y^2 w + \mathcal{O}(\varepsilon^2).$$

Therefore

$$w(t+1) - w(t) \approx \varepsilon yx - \varepsilon y^2 w = \varepsilon y(x - yw).$$

Step 6. Read off the componentwise rule. For coordinate j, the update is

$$\Delta w_j = \varepsilon y(x_j - y w_j).$$

This is Oja's rule. The Hebbian term $+\varepsilon y x_j$ pulls the weight toward the current input. The normalization term $-\varepsilon y^2 w_j$ subtracts a small amount in the current weight direction, exactly the direction that would otherwise make the norm grow.

The derivation also explains why Oja's rule is local and online. It does not require computing the full covariance matrix. It only needs the current input coordinate x_j, the current output y, the current weight w_j, and a small learning rate.

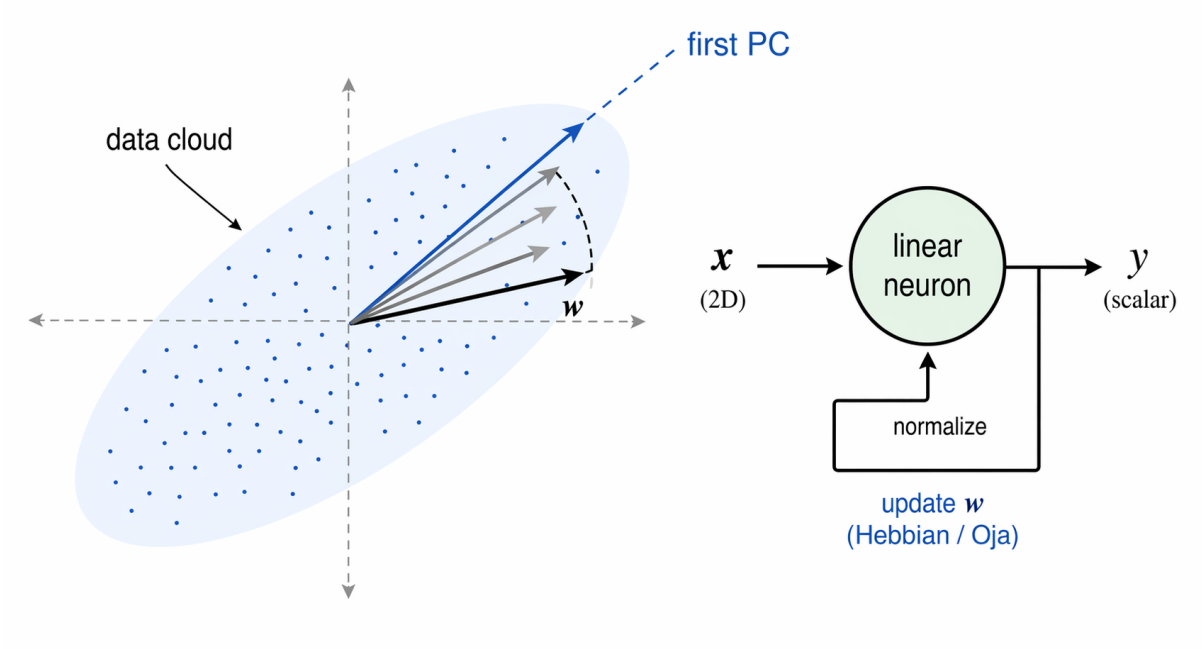

Figure 4: Online PCA with one linear neuron. Hebbian updates rotate the weight vector toward the high-variance direction of the centered data cloud. Oja's normalization keeps the vector length bounded, so the stable direction is the first principal component.

2.6 The Fixed Point Intuition

At convergence under Oja's rule, w is a unit vector aligned with the leading eigenvector. A convergence point is a direction where the average update is zero: on average, learning no longer moves the vector. This happens when Hebbian growth is exactly balanced by the normalization term. If $w = e_1$, then

$$\mathbb{E}[yx] = Cw = \lambda_1 w,$$

and

$$\mathbb{E}[y^2] = w^\mathsf{T} C w = \lambda_1.$$

Thus the mean update becomes

$$\varepsilon(Cw - \mathbb{E}[y^2]w) = \varepsilon(\lambda_1 w - \lambda_1 w) = 0.$$

The leading eigenvector is stable because any small component along a lower-variance direction grows more slowly than the leading component. If the vector is nudged slightly away from e_1, the dynamics amplify the e_1 part more strongly and pull the direction back.

2.7 From One Component To Many: Sanger's Rule

One neuron learns one principal direction. If several neurons all use the same Oja rule on the same input, they all have the same favorite solution: the first principal component. To learn several directions, the system needs multiple neurons and a mechanism that prevents them all from converging to the same first component.

Let

$$y_i = w_i^\mathsf{T} x$$

be the output of neuron i. Sanger's generalized Hebbian algorithm uses

$$\Delta w_i = \varepsilon y_i \left(x - \sum_{k=1}^{i} y_k w_k \right).$$

Componentwise this is

$$\Delta w_{ij} = \varepsilon y_i \left(x_j - \sum_{k=1}^{i} w_{kj} y_k \right).$$

For $i = 1$, this is Oja's rule. For $i = 2$, the input is first stripped of the component already explained by w_1. For $i = 3$, the input is stripped of the components explained by w_1 and w_2, and so on.

This is what orthogonalization means here. A later neuron does not learn from the full input; it learns from the residual, the part of the input that cannot already be reconstructed from earlier neurons. Gram-Schmidt is the classical procedure that turns a list of vectors into perpendicular directions by subtracting projections onto directions already chosen. Sanger's rule performs the same subtraction online, while data are arriving.

Each neuron performs Oja-like learning on the residual left after earlier neurons have taken their directions. Under the usual assumptions, the weights converge to the ordered principal components:

$$w_1 \to e_1, \qquad w_2 \to e_2, \qquad \ldots, \qquad w_M \to e_M.$$

After learning, the neuron outputs are the PCA scores:

$$y_i = e_i^\mathsf{T} x.$$

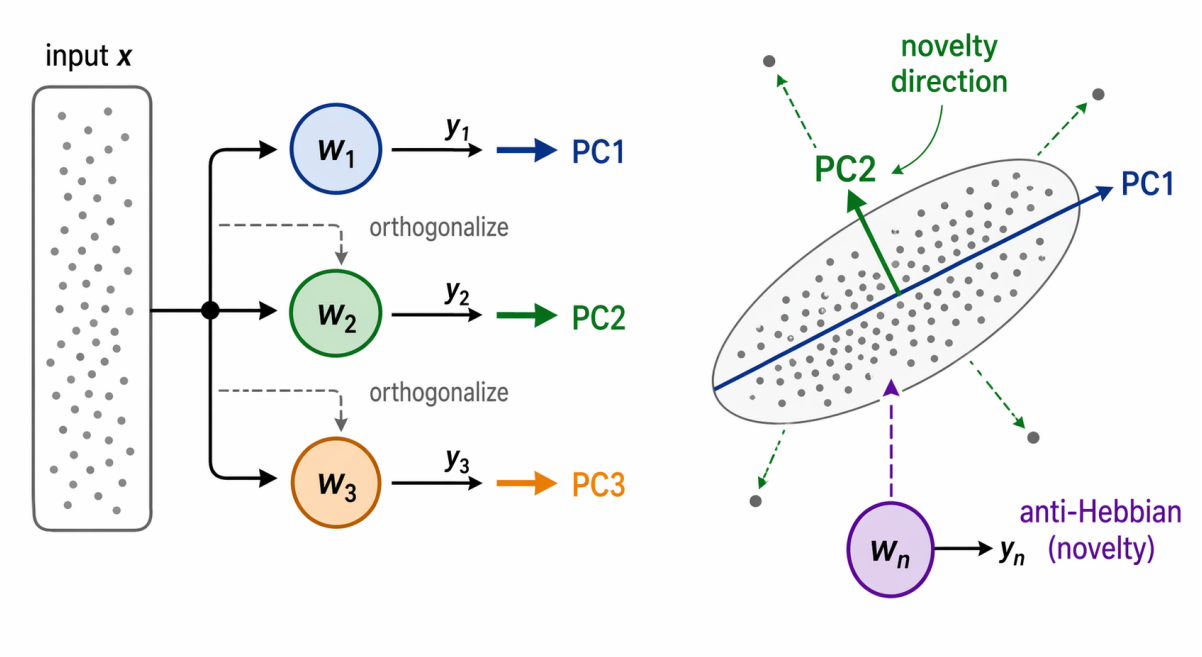

Figure 5: Multiple Hebbian neurons need competition or orthogonalization. Sanger's rule makes each later neuron learn from the residual left by earlier neurons, producing ordered principal components. Reversing the sign gives an anti-Hebbian novelty direction aligned with low-variance structure.

2.8 Novelty Filters And Anti-Hebbian Learning

Large-variance directions describe what is common. Small-variance directions can reveal what is surprising. For example, suppose normal data mostly lie near a line or a plane. Motion along that line or plane is ordinary, but motion away from it is unusual. The low-variance direction points away from the normal data shape.

The lecture expresses this with the last principal component. After PCA learning,

$$y = e_N^\top x$$

is the projection onto the direction with the smallest eigenvalue. A large absolute value of y signals that the input has energy in a direction where normal data vary little. This is the novelty-filter idea: the output is small for typical inputs and large when the input violates the learned regularity.

The online rule is anti-Hebbian:

$$\Delta w = -\varepsilon y x.$$

Anti-Hebbian means that co-activation weakens rather than strengthens a connection. If an input direction repeatedly produces a large output, the weight vector is pushed away from that direction.

Averaging gives

$$\mathbb{E}[\Delta w] = -\varepsilon C w.$$

In the eigenbasis,

$$\Delta a_j = -\varepsilon \lambda_j a_j.$$

Large-eigenvalue components decay fastest because their λ_j values are largest. With normalization, the remaining direction is the eigenvector with the smallest eigenvalue. A normalized anti-Hebbian version of Oja's rule therefore converges to e_N.

2.9 Connection To Total Least Squares

The novelty-filter direction also appears in linear regression geometry. A residual is the error left after a model tries to explain a point. Ordinary least squares minimizes vertical residuals and is appropriate when the x-coordinate is treated as exact and the noise is only in the dependent variable. If both coordinates are noisy, a more symmetric objective is total least squares: minimize orthogonal distances to a line.

For centered two-dimensional data, the normal vector w to the best-fit line solves

$$\min_{\|w\|=1} w^{\mathsf{T}} C w.$$

This is the opposite of the first PCA objective. Maximizing $w^{\mathsf{T}} C w$ gives the high-variance direction along the data. Minimizing it gives the low-variance normal direction. The solution is the normalized eigenvector of C with the smallest eigenvalue.

2.10 Summary

Hebbian learning turns PCA from an offline spectral computation into an online dynamical process. Pure Hebbian learning performs covariance multiplication and therefore points toward the first principal component, but its norm diverges. Oja's rule adds normalization and stabilizes the first component. Sanger's rule adds sequential orthogonalization so multiple neurons learn ordered principal components. Anti-Hebbian learning reverses the direction of the dynamics and, with normalization, finds low-variance novelty directions.

3 Lecture 3: Hidden Markov Models

Hidden Markov Models (HMMs) are the first explicitly sequential latent-variable models in the course. The core problem is simple to state: we observe a time series

$$x^{(1)}, x^{(2)}, \ldots, x^{(T)},$$

and we want a probabilistic model for the whole sequence rather than for isolated samples.

The difficulty is that time series are not i.i.d. If the current observation depends on the past, then the likelihood does not factorize as

$$P\big(x^{(1)}, \ldots, x^{(T)} \mid w\big) \neq \prod_{t=1}^{T} P\big(x^{(t)} \mid w\big).$$

The left-hand side is a joint model for the whole trajectory. The right-hand side would be valid only for independent observations. Sequential dependence therefore changes both the geometry of the model and the optimization problem used to fit it.

A useful mental model is this: an HMM assumes that the observed signal is generated by an underlying discrete dynamical system. The hidden state stores the relevant context, and the observation is a noisy readout of that state. The model is therefore a bridge between clustering and dynamics: each state behaves like a mixture component, but the components are linked across time by a Markov chain.

3.1 Why plain Markov models are not enough

A first-order Markov chain for the observed variables would assume

$$P\big(x^{(t)} \mid x^{(1)}, \dots, x^{(t-1)}\big) \approx P\big(x^{(t)} \mid x^{(t-1)}\big).$$

Under this assumption the full sequence probability becomes

$$P\big(x^{(1)}, \dots, x^{(T)}\big) = P\big(x^{(1)}\big) \prod_{t=2}^{T} P\big(x^{(t)} \mid x^{(1)}, \dots, x^{(t-1)}\big) \approx P\big(x^{(1)}\big) \prod_{t=2}^{T} P\big(x^{(t)} \mid x^{(t-1)}\big).$$

This reduces the complexity of the model, but it can be too restrictive because the current observation may depend on a longer latent context than the immediately previous observation reveals.

Increasing the observed Markov order to K gives

$$P\big(x^{(t)} \mid x^{(1)}, \dots, x^{(t-1)}\big) \approx P\big(x^{(t)} \mid x^{(t-1)}, x^{(t-2)}, \dots, x^{(t-K)}\big).$$

This makes the model more flexible, but the number of parameters can grow rapidly with K. HMMs solve this trade-off differently: instead of keeping a long memory directly in the observations, they compress the relevant past into a hidden state.

3.2 From mixture models to hidden state dynamics

Mixture models already introduce latent variables. For independent samples, one draws a hidden component label and then generates an observation from the corresponding component density. HMMs keep the same emission idea but add temporal dependence between successive latent labels.

Let

$$m^{(t)} \in \{e_1, \dots, e_M\}$$

be a hidden state encoded in a 1-of-M representation. Here e_q denotes the qth standard basis vector, so the event "the system is in state q at time t" means $m_q^{(t)} = 1$.

The HMM assumptions are:

$$P\big(m^{(t)} \mid m^{(1)}, \dots, m^{(t-1)}\big) = P\big(m^{(t)} \mid m^{(t-1)}\big),$$
$$P\big(x^{(t)} \mid x^{(1)}, \dots, x^{(t-1)}, m^{(1)}, \dots, m^{(t)}\big) = P\big(x^{(t)} \mid m^{(t)}\big).$$

The first equation is the Markov property in hidden-state space. The second is the conditional independence assumption: once the current state is known, the current observation no longer needs the rest of the past.

The figure makes the modeling choice explicit. Horizontal arrows encode the dynamics of the latent state. Vertical arrows encode the observation model. Because the latent state is discrete and low-dimensional, the model can retain long-range dependence in the observed signal without requiring a high-order observed Markov chain.

3.3 Transition model, initial distribution, and emissions

The hidden-state dynamics are parameterized by a transition matrix

$$A \in [0,1]^{M \times M}, \qquad A_{qr} = P\big(m_r^{(t)} = 1 \mid m_q^{(t-1)} = 1\big),$$

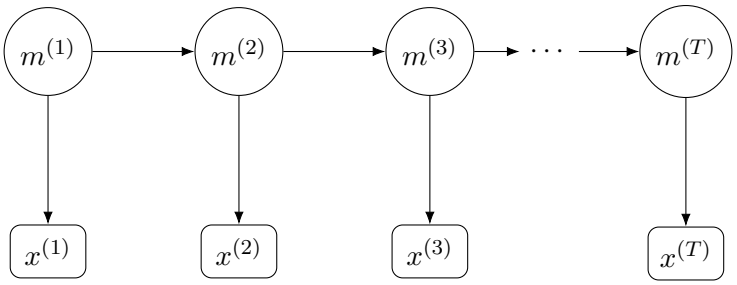

Figure 6: A Hidden Markov Model separates dynamics and readout. The hidden chain $m^{(1)} \to \cdots \to m^{(T)}$ stores the retained state variables, while each observation $x^{(t)}$ is emitted only from the current hidden state.

with row-wise normalization

$$\sum_{r=1}^{M} A_{qr} = 1 \qquad \text{for each } q = 1, \ldots, M.$$

Each row describes where the process can move next when it is currently in state q. Varying a row changes the dynamical persistence and switching behavior of that state. Large diagonal entries favor persistence; large off-diagonal entries favor frequent transitions.

The initial hidden state is drawn from a probability vector

$$b \in [0,1]^M, \qquad b_q = P(m_q^{(1)} = 1), \qquad \sum_{q=1}^{M} b_q = 1.$$

The vector b determines how sequences start before any transition has occurred.

The emission model specifies the observation distribution conditioned on the current hidden state. For a general family with state-specific parameters

$$\phi = \{\phi_1, \ldots, \phi_M\},$$

we write

$$P(x^{(t)} \mid m^{(t)}, \phi) = \prod_{q=1}^{M} P(x^{(t)} \mid \phi_q)^{m_q^{(t)}}.$$

Because exactly one entry of $m^{(t)}$ equals 1, this product simply selects the emission density of the active state.

Two standard examples from the lecture are worth keeping separate:

- **Gaussian emissions.** For continuous observations $x \in \mathbb{R}^N$,

  $$P(x \mid m_q = 1, \phi) = \mathcal{N}(x \mid \mu_q, \Sigma_q).$$

  The mean μ_q is the state-specific typical observation, while Σ_q measures how much variability is still present after conditioning on state q.

- **Discrete emissions.** For categorical observations encoded as $x \in \{0,1\}^N$ with $\sum_i x_i = 1$,

  $$P(x \mid m_q = 1, \phi) = \prod_{i=1}^{N} \phi_{qi}^{x_i}, \qquad \sum_{i=1}^{N} \phi_{qi} = 1.$$

  Here ϕ_{qi} is the probability that state q emits symbol i.

3.4 Joint distribution of the HMM

Putting initial state, transitions, and emissions together gives the joint distribution of hidden and observed variables:

$$P\big(\{x^{(t)}\},\{m^{(t)}\} \mid w\big) = P\big(m^{(1)} \mid b\big) \prod_{t=2}^{T} P\big(m^{(t)} \mid m^{(t-1)}, A\big) \prod_{t=1}^{T} P\big(x^{(t)} \mid m^{(t)}, \phi\big),$$

where

$$w = \{A, b, \phi\}$$

collects all model parameters.

This factorization is the algebraic heart of the model. It says that an HMM is a generative story with three ingredients: how the hidden process starts, how it moves, and how each hidden state emits an observation. Every algorithm in the lecture uses this factorization, either to sum over hidden paths or to maximize over them.

A subtle but important point is that the observed process need not itself be Markov of any fixed low order. Even when the hidden state is first-order Markov, the marginalized observation sequence can have richer dependence because the hidden state carries forward information not directly visible in the current observation.

3.5 Sampling from the model

The generative procedure is straightforward:

1. choose parameters A, b, and ϕ;
2. sample $m^{(1)} \sim P(m^{(1)} \mid b)$;
3. sample $x^{(1)} \sim P(x^{(1)} \mid m^{(1)}, \phi)$;
4. for $t = 2, \ldots, T$, sample

$$m^{(t)} \sim P\big(m^{(t)} \mid m^{(t-1)}, A\big), \qquad x^{(t)} \sim P\big(x^{(t)} \mid m^{(t)}, \phi\big).$$

This view is useful practically. It clarifies what an HMM can express and what it cannot. The model can represent persistent hidden regimes, switching between regimes, and state-specific observation statistics. It cannot represent arbitrary memory unless that memory can be compressed into the chosen discrete state space.

3.6 A tiny worked example before the full algorithms

Before introducing Baum–Welch and Viterbi, it helps to run one tiny HMM by hand. Suppose the hidden state is the weather,

$$m^{(t)} \in \{\text{sunny}, \text{rainy}\},$$

and the observation is whether we see an umbrella. Consider the parameters

$$b = \begin{bmatrix} 0.6 & 0.4 \end{bmatrix}, \qquad A = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix},$$

where the first row corresponds to *sunny* and the second to *rainy*. The emission probabilities are

$$P(\text{umbrella} \mid \text{sunny}) = 0.1, \qquad P(\text{umbrella} \mid \text{rainy}) = 0.9.$$

So umbrellas are much more likely on rainy days, but not impossible on sunny days.

Now suppose the first observation is *umbrella*. The forward initialization becomes

$$\alpha^{(1)}_{\text{sunny}} = 0.6 \cdot 0.1 = 0.06, \qquad \alpha^{(1)}_{\text{rainy}} = 0.4 \cdot 0.9 = 0.36.$$

Even before normalization, the message already says that rainy is much more plausible after seeing one umbrella. If we normalize,

$$P(m^{(1)} = \text{sunny} \mid x^{(1)} = \text{umbrella}) = \frac{0.06}{0.06 + 0.36} \approx 0.143,$$

so the posterior belief in *rainy* is about 0.857.

Suppose the second observation is again *umbrella*. The next forward step is

$$\alpha^{(2)}_{\text{sunny}} = 0.1\big(0.06 \cdot 0.8 + 0.36 \cdot 0.3\big) = 0.0156,$$

$$\alpha^{(2)}_{\text{rainy}} = 0.9\big(0.06 \cdot 0.2 + 0.36 \cdot 0.7\big) = 0.2376.$$

So a second umbrella makes the rainy explanation even more dominant. This tiny calculation shows the whole logic of HMM inference: the hidden belief is updated by combining *state persistence* (the transition matrix) with *how compatible each state is with the new observation* (the emission model).

The sleep-state example in the lecture is mathematically the same pattern. The hidden states are no longer sunny and rainy, but sleep stages; the observations are no longer umbrellas, but physiological feature vectors. The algorithms do not change, only the interpretation of states and emissions becomes more domain-specific.

3.7 Learning by maximum likelihood: the Baum–Welch algorithm

Given observations alone, the parameter estimation problem is

$$w^* = \arg\max_w P\big(\{x^{(t)}\} \mid w\big).$$

Since the hidden states are unobserved, the data likelihood requires marginalization:

$$P\big(\{x^{(t)}\} \mid w\big) = \sum_{\{m^{(t)}\}} P\big(\{x^{(t)}\}, \{m^{(t)}\} \mid w\big).$$

The sum runs over all hidden state sequences. Its size grows as M^T, so direct evaluation is infeasible except for tiny toy problems.

The lecture therefore uses Expectation–Maximization (EM). For HMMs, the EM specialization is called the *Baum–Welch algorithm*. As in other latent-variable models, EM alternates between computing posterior responsibilities for latent variables and maximizing the expected complete-data log-likelihood.

3.8 E-step: posterior state occupancies and transitions

For fixed old parameters w_{old}, define the posterior state occupancy

$$\gamma(m^{(t)}) = P(m^{(t)} \mid \{x^{(s)}\}, w_{\text{old}}),$$

and in particular

$$\gamma(m_q^{(t)}) = P(m_q^{(t)} = 1 \mid \{x^{(s)}\}, w_{\text{old}}).$$

This is the posterior probability that hidden state q was active at time t.

Similarly, define the posterior transition probability

$$\xi(m^{(t-1)}, m^{(t)}) = P(m^{(t-1)}, m^{(t)} \mid \{x^{(s)}\}, w_{\text{old}}),$$

and componentwise

$$\xi(m_q^{(t-1)}, m_r^{(t)}) = P(m_q^{(t-1)} = 1, m_r^{(t)} = 1 \mid \{x^{(s)}\}, w_{\text{old}}).$$

This measures how much posterior mass is assigned to a transition from state q to state r between times $t-1$ and t.

With these quantities the EM auxiliary function becomes

$$\begin{aligned} Q(w, w_{\text{old}}) = & \sum_{q=1}^{M} \gamma(m_q^{(1)}) \ln b_q \\ & + \sum_{t=2}^{T} \sum_{q=1}^{M} \sum_{r=1}^{M} \xi(m_q^{(t-1)}, m_r^{(t)}) \ln A_{qr} \\ & + \sum_{t=1}^{T} \sum_{q=1}^{M} \gamma(m_q^{(t)}) \ln P(x^{(t)} \mid \phi_q). \end{aligned}$$

Each term has a clear interpretation. The first counts how often a state starts a sequence. The second counts expected transitions. The third counts expected emissions. EM therefore turns latent-sequence learning into weighted maximum likelihood for initial probabilities, transitions, and state-specific emission models.

3.9 Forward–backward recursion

The E-step is tractable because γ and ξ can be computed by dynamic programming rather than by summing over all hidden paths explicitly.

Define the forward variable

$$\alpha(m^{(t)}) = P(x^{(1)}, \ldots, x^{(t)}, m^{(t)} \mid w_{\text{old}}).$$

It is the joint probability of the partial observation prefix and the current hidden state. For component q,

$$\alpha_q^{(t)} := \alpha(m_q^{(t)} = 1).$$

The initialization is

$$\alpha_q^{(1)} = b_q \, P(x^{(1)} \mid \phi_q).$$

For $t \geq 2$ the recursion is

$$\alpha_r^{(t)} = P\big(x^{(t)} \mid \phi_r\big) \sum_{q=1}^{M} \alpha_q^{(t-1)} A_{qr}.$$

The inner sum transports probability mass through the hidden dynamics; the emission term then scores how well state r explains the new observation.

Define the backward variable

$$\beta\big(m^{(t)}\big) = P\big(x^{(t+1)}, \ldots, x^{(T)} \mid m^{(t)}, w_{\text{old}}\big),$$

with component notation $\beta_q^{(t)} := \beta(m_q^{(t)} = 1)$. The endpoint is

$$\beta_q^{(T)} = 1,$$

because after time T no observations remain to be explained. The recursion backward in time is

$$\beta_q^{(t)} = \sum_{r=1}^{M} A_{qr} P\big(x^{(t+1)} \mid \phi_r\big) \beta_r^{(t+1)}.$$

This has the complementary interpretation: starting from state q at time t, it sums over all possible next states and future evidence.

Once α and β are known, posterior occupancies follow as

$$\gamma\big(m_q^{(t)}\big) = \frac{\alpha_q^{(t)} \beta_q^{(t)}}{P\big(\{x^{(s)}\} \mid w_{\text{old}}\big)}.$$

Likewise,

$$\xi\big(m_q^{(t-1)}, m_r^{(t)}\big) = \frac{\alpha_q^{(t-1)} A_{qr} P\big(x^{(t)} \mid \phi_r\big) \beta_r^{(t)}}{P\big(\{x^{(s)}\} \mid w_{\text{old}}\big)}.$$

The denominator is the sequence likelihood. In many M-step ratios it cancels, which is why the algorithm can be implemented with normalized messages.

Complexity. The forward and backward passes each require $\mathcal{O}(M^2 T)$ operations. This is the main computational gain of the HMM factorization: exponential summation over hidden paths is replaced by polynomial-time dynamic programming.

3.10 M-step updates

The M-step maximizes $Q(w, w_{\text{old}})$ subject to normalization constraints. For the initial distribution,

$$b_q^{\text{new}} = \gamma\big(m_q^{(1)}\big).$$

So the new initial-state probability is simply the posterior probability that the sequence started in state q.

For the transition matrix,

$$A_{qr}^{\text{new}} = \frac{\sum_{t=2}^{T} \xi\big(m_q^{(t-1)}, m_r^{(t)}\big)}{\sum_{t=2}^{T} \sum_{s=1}^{M} \xi\big(m_q^{(t-1)}, m_s^{(t)}\big)}.$$

The numerator is the expected number of transitions from q to r. The denominator is the expected number of times state q was left. Thus each updated row is a normalized expected transition count.

The emission update depends on the chosen family:

- **Gaussian emissions with scalar variance.**

$$\mu_q^{\text{new}} = \frac{\sum_{t=1}^{T} \gamma(m_q^{(t)}) x^{(t)}}{\sum_{t=1}^{T} \gamma(m_q^{(t)})},$$

$$\sigma_{q,\text{new}}^2 = \frac{\sum_{t=1}^{T} \gamma(m_q^{(t)}) \big(x^{(t)} - \mu_q^{\text{new}}\big)^2}{\sum_{t=1}^{T} \gamma(m_q^{(t)})}.$$

  The mean is a posterior-weighted average of observations assigned to state q. The variance is the corresponding posterior-weighted residual variance.

- **Discrete emissions.**

$$\phi_{qi}^{\text{new}} = \frac{\sum_{t=1}^{T} \gamma(m_q^{(t)}) x_i^{(t)}}{\sum_{t=1}^{T} \gamma(m_q^{(t)})}.$$

  This is the posterior-weighted frequency with which state q emits symbol i.

The practical implication is simple: once the hidden-state posteriors are available, the M-step looks like re-estimating a mixture model with soft assignments, except that the assignments now respect temporal structure through ξ as well as γ.

3.11 Likelihood monitoring and numerical stability

The sequence likelihood can be recovered from the forward–backward variables as

$$P\big(\{x^{(s)}\} \mid w\big) = \sum_{q=1}^{M} \alpha_q^{(t)} \beta_q^{(t)} \qquad \text{for any } t.$$

Monitoring this likelihood is useful because EM should not decrease it from one iteration to the next.

However, raw forward and backward messages multiply many small probabilities, so underflow is a serious issue for long sequences. The standard cure is to use normalized or scaled messages. One keeps track of scaling constants at each time step, updates normalized versions of α and β, and reconstructs the log-likelihood from the scaling terms. The model itself does not change; only the numerics do.

3.12 Several training sequences

The lecture emphasizes that HMMs do not need to be trained on a single long sequence only. If we observe multiple sequences

$$\big\{x^{(1,n)}, \ldots, x^{(T_n,n)}\big\}, \qquad n = 1, \ldots, p,$$

we run the forward–backward recursion independently on each sequence and then sum the sufficient statistics across sequences before performing the M-step. The formulas stay the same, but expected counts and weighted residuals are aggregated over n as well as over t.

This is the correct bridge from the single-sequence derivation to practical datasets, where each training example is often a separate trajectory, utterance, trial, or episode.

3.13 Inference of hidden states: the Viterbi algorithm

Baum–Welch estimates parameters by averaging over all hidden paths. Sometimes that is not the end goal. In speech recognition, segmentation, or sleep staging, we often want the single most likely hidden-state sequence for a fixed trained model.

That decoding problem is

$$\{m_*^{(t)}\} = \arg\max_{\{m^{(t)}\}} P\big(\{m^{(t)}\} \mid \{x^{(t)}\}, w\big).$$

Since the denominator $P(\{x^{(t)}\} \mid w)$ does not depend on the hidden path, this is equivalent to maximizing the joint probability

$$\arg\max_{\{m^{(t)}\}} P\big(\{m^{(t)}\}, \{x^{(t)}\} \mid w\big).$$

Taking logs converts products into sums and improves numerical stability:

$$\arg\max_{\{m^{(t)}\}} \ln P\big(\{m^{(t)}\}, \{x^{(t)}\} \mid w\big).$$

The resulting optimization can be read as a longest-path problem on a trellis whose node scores are log emission probabilities and whose edge scores are log transition probabilities.

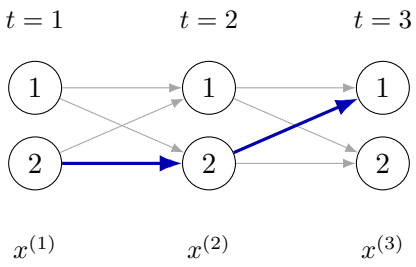

Figure 7: Viterbi decoding searches a trellis of possible hidden-state paths. Each node corresponds to a state at a time step, each arrow contributes a transition score, and the highlighted chain illustrates one candidate best path recovered by dynamic programming and backtracking.

3.14 Dynamic programming for the best path

Define

$$\delta_q^{(t)} = \max_{m^{(1)},\dots,m^{(t-1)}} \ln P\big(m^{(1)}, \dots, m^{(t-1)}, m_q^{(t)} = 1, x^{(1)}, \dots, x^{(t)} \mid w\big).$$

This is the log probability of the best partial path that ends in state q at time t.

Initialization is

$$\delta_q^{(1)} = \ln b_q + \ln P\big(x^{(1)} \mid \phi_q\big).$$

For $t \geq 2$ the recursion is

$$\delta_q^{(t)} = \max_{r=1,\dots,M} \left\{ \delta_r^{(t-1)} + \ln A_{rq} + \ln P\big(x^{(t)} \mid \phi_q\big) \right\}.$$

The best path to state q at time t must come from some predecessor state r. Dynamic programming works because only the best predecessor matters for future optimization.

To recover the path, one stores the maximizing predecessor

$$\psi_q^{(t)} = \arg\max_{r=1,\dots,M} \left\{ \delta_r^{(t-1)} + \ln A_{rq} + \ln P\big(x^{(t)} \mid \phi_q\big) \right\}.$$

After the forward pass,

$$q_*^{(T)} = \arg\max_{q=1,\dots,M} \delta_q^{(T)}$$

selects the final state of the best path. One then backtracks through the stored predecessor indices $\psi_q^{(t)}$ to reconstruct the entire hidden-state sequence.

Posterior marginals versus most likely path. It is worth separating two different inference questions. The marginal posterior $\gamma(m_q^{(t)})$ asks how likely state q is at time t after averaging over all full paths. Viterbi decoding asks for one globally most probable path. These answers can differ. A state may have the highest marginal probability at a given time without belonging to the single best global path.

3.15 One-step-ahead prediction

Once the filtered belief over the current hidden state is known, the HMM also gives a predictive distribution for the next observation:

$$P\big(x^{(t+1)} \mid x^{(1)}, \dots, x^{(t)}\big) = \sum_{m^{(t)}, m^{(t+1)}} P\big(x^{(t+1)} \mid m^{(t+1)}\big) P\big(m^{(t+1)} \mid m^{(t)}\big) P\big(m^{(t)} \mid x^{(1)}, \dots, x^{(t)}\big).$$

This formula reveals the full computational pattern of state-space models: infer the current latent state, propagate it forward through the dynamics, then map the resulting latent prediction into observation space.

The student should notice the conceptual split. Filtering updates uncertainty about the present hidden state. Prediction pushes that uncertainty one step into the future.

3.16 Example: sleep-state detection

The sleep-stage example in the lecture makes the abstract model concrete. Physiological recordings are first converted into feature vectors on fixed 30-second epochs. The hidden states represent sleep stages such as wakefulness, REM, or slow-wave sleep. The observation vectors are then modeled as emissions from these hidden states.

Two modeling choices matter especially in this application:

- **Constrained transitions.** Not every sleep-stage switch is physiologically plausible. Setting selected entries of A to zero encodes impossible or extremely unlikely transitions directly into the dynamics.
- **Discrete observation model after clustering.** Continuous feature vectors can first be clustered, for example by K-means, and the resulting cluster labels can then be used as discrete HMM observations. This turns a complex continuous signal into a symbolic sequence that is easier to model with a discrete emission matrix.

This example illustrates the real practical role of HMMs: they are most effective when some domain knowledge is available about hidden regimes, admissible transitions, and useful observation features.

3.17 Remarks and practical limits

Several lecture remarks are worth collecting in one place.

- **Regularization and priors.** One can place priors on b, A, and ϕ, leading to maximum-a-posteriori rather than pure maximum-likelihood estimation.
- **Emission-family dependence.** EM updates are especially convenient when the emission model is discrete or belongs to the exponential family, because closed-form weighted maximum-likelihood updates are then available.
- **Continuous hidden states.** Replacing the discrete hidden state by a real-valued state leads toward linear-Gaussian state-space models and Kalman filtering.
- **Model selection.** The number of hidden states, the choice of features, and the emission model class are not minor details. They determine what the latent state can retain and what structure the model can explain.

3.18 Summary

An HMM models sequential data with a discrete hidden state that evolves as a first-order Markov chain and emits observations through a state-specific distribution. The key factorization is

$$P(\{x^{(t)}\}, \{m^{(t)}\} \mid w) = P(m^{(1)} \mid b) \prod_{t=2}^{T} P(m^{(t)} \mid m^{(t-1)}, A) \prod_{t=1}^{T} P(x^{(t)} \mid m^{(t)}, \phi).$$

This turns sequence modeling into a latent-variable problem with structured dependencies. Baum–Welch uses forward–backward recursions to compute posterior occupancies and expected transitions, then updates initial probabilities, transitions, and emissions by weighted maximum likelihood. Viterbi decoding uses a max-product version of the same dynamic-programming idea to extract the single most likely hidden path.

Practically, the student should now be able to do three things: write down the HMM factorization for a concrete problem, compute or at least interpret the forward–backward quantities α, β, γ, and ξ, and decide whether the task requires averaging over hidden paths for learning or maximizing over hidden paths for decoding.

4 Mathematical Synthesis: Spectral Structure and Online Learning

Machine Intelligence II begins from unsupervised structure: what can be learned when the data arrive without labels? Principal component analysis is the clean prototype because every step is visible. Center the data, measure covariance, diagonalize the covariance, project onto the strongest directions, and track exactly what variance was kept or discarded. Hebbian learning then asks how the same directions can be found online, one sample at a time.

4.1 The Covariance Matrix Is A Geometry

Let centered data vectors be rows of $X \in \mathbb{R}^{n \times d}$. The empirical covariance is

$$\Sigma = \frac{1}{n} X^{\top} X.$$

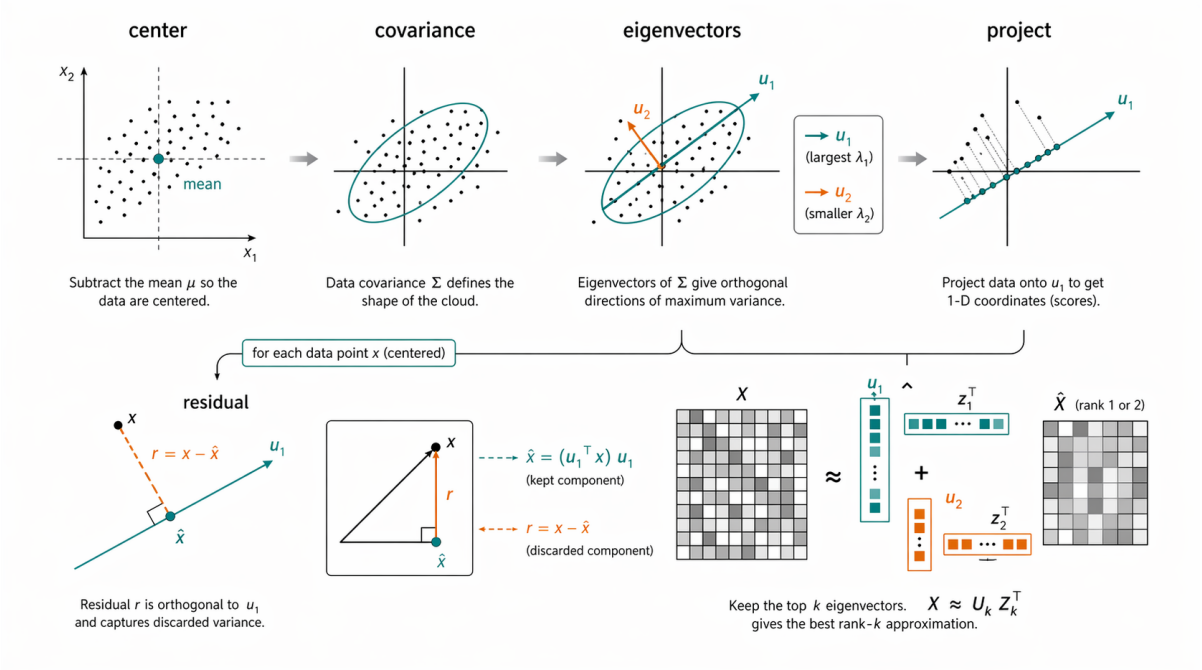

Figure 8: PCA as spectral compression. Centering removes the mean, the covariance ellipse reveals directions of variation, eigenvectors provide orthogonal coordinates, projection keeps high-variance coordinates, and residuals are the discarded orthogonal variation.

For any unit direction u, the variance of the data after projection onto u is

$$\frac{1}{n}\|Xu\|^2 = u^\top \Sigma u.$$

So PCA asks for the direction with maximal quadratic form:

$$u_1 \in \arg\max_{\|u\|=1} u^\top \Sigma u.$$

The solution is the leading eigenvector of Σ. The corresponding eigenvalue is the variance captured along that direction.

4.2 Projection And Reconstruction

If $U_k = [u_1, \ldots, u_k]$ contains the top k orthonormal eigenvectors, then the low-dimensional score vector for a data point x is

$$z = U_k^\top x,$$

and the rank-k reconstruction is

$$\hat{x} = U_k z = U_k U_k^\top x.$$

The residual is

$$r = x - \hat{x}.$$

It is orthogonal to the retained subspace, so the reconstruction separates data into an explained component and a discarded component.

Variance captured.

$$\text{captured variance}(k) = \sum_{j=1}^{k} \lambda_j, \qquad \text{fraction captured} = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{d} \lambda_j}.$$

The spectrum $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$ is a diagnostic for intrinsic dimension. A fast spectral decay means the data are close to a low-dimensional subspace.

4.3 Online Learning Turns Spectra Into Dynamics

For a linear neuron $y = w^\top x$, the basic Hebbian update is

$$\Delta w = \varepsilon y x.$$

Averaged over centered samples, this becomes

$$\mathbb{E}[\Delta w] = \varepsilon \mathbb{E}[x x^\top] w = \varepsilon C w.$$

So Hebbian learning is a stochastic power method. Components of w along high-eigenvalue directions grow faster than components along low-eigenvalue directions. The direction of w therefore approaches the first principal component, but the norm grows without bound unless the learning rule includes a constraint.

Oja's rule adds that constraint:

$$\Delta w = \varepsilon y (x - y w).$$

The first term is Hebbian attraction toward directions that explain variance. The second term is a local normalization pressure. Together they make the unit vector $w/\|w\|$ converge to the leading principal direction.

4.4 Why PCA Is The Right Baseline

PCA is linear, unsupervised, and global. Those limitations are also why it is such a useful baseline. If a nonlinear representation method improves on PCA, it should be clear which PCA assumption it breaks: linearity, globality, Euclidean reconstruction error, or Gaussian-like covariance structure.

The mathematical virtue of PCA is that it solves two problems at once. It maximizes retained variance and minimizes squared reconstruction error:

$$\min_{\operatorname{rank}(\hat{X}) \leq k} \|X - \hat{X}\|_F^2.$$

By the Eckart–Young theorem, the solution is the truncated singular value decomposition. This makes PCA the reference point for later factor models, autoencoders, manifold learning, and probabilistic latent-variable models.